S2D-01: Splunk to Dynatrace Migration - Getting Started

> **Series:** S2D — Splunk to Dynatrace Migration | **Notebook:** 1 of 9 | **Created:** January 2026 | **Last Updated:** 11/11/2026

Overview

This notebook series provides comprehensive guidance for migrating monitoring capabilities from Splunk to Dynatrace. Whether you're moving dashboards, alerts, reports, or log queries, these notebooks will help you understand the key differences between platforms and make informed translation decisions.

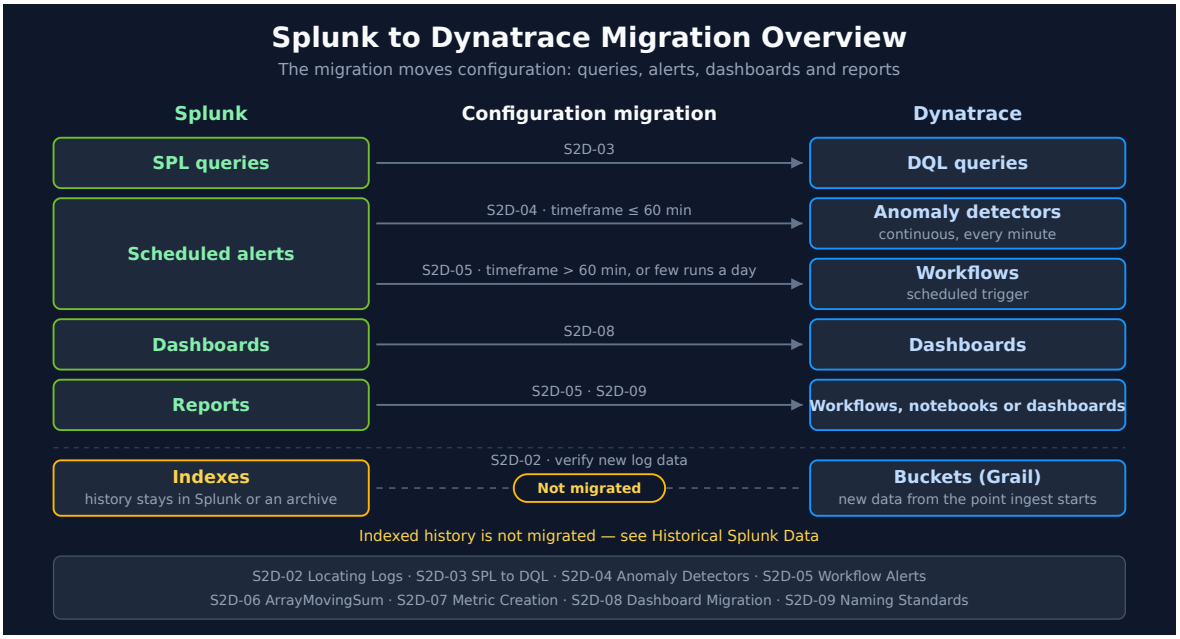


Table of Contents

1. Key Platform Differences

2. Migration Planning
3. Series Roadmap
4. Quick Start: Verify Your First Log Source

Sprint 1.337 (April 2026) Updates Affecting S2D

Three 1.337 release items touch the Splunk → Dynatrace migration:

1. **Primary fields on AWS Lambda logs (OneAgent 1.337)** — *"Primary fields set via DT_TAGS are now added to logs and spans in AWS Lambda deployments of OneAgent."* The item covers Lambda deployments only, not hosts in general. Where it applies, you do not need an OpenPipeline processor to add those fields to Lambda logs.
2. **OpenPipeline field recommendations (SaaS 1.337)** — *"Recommendations automatically cover all permission- and cost-relevant fields or dimensions, with support for all Smartscape identifiers and Grail primary tags."* This helps when you convert Splunk field extractions into OpenPipeline processors.
3. **Configuration API deprecations (SaaS 1.337)** — *"Certain Configuration API endpoints are now deprecated"*, and *"No immediate action is required on your part."* The release note adds *"We recommend migrating to the Environment API v2 endpoints for a more robust integration"*, so point new migration automation there. If that automation uses platform tokens, they carry the `dt0s16` prefix: *"dt0s16 Platform Token enabling programmatic access to Dynatrace platform services."*

These changes reinforce the Gen3-first migration baseline.

> **Sources:** OneAgent 1.337 (DT docs), SaaS 1.337 (DT docs), Tokens and authentication (DT docs).

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Grail enabled

Requirement	Details
Permissions	`storage:logs:read`, `storage:buckets:read`, `settings:objects:read`
Splunk Access	Ability to view existing queries, alerts, dashboards
Knowledge	Basic familiarity with both Splunk SPL and Dynatrace DQL

Learning Objectives

By the end of this series, you will be able to:

1. Validate that required log data is available in Dynatrace
2. Translate SPL queries to DQL
3. Migrate Splunk alerts to Anomaly Detectors or Workflows
4. Convert Splunk dashboards to Dynatrace format
5. Apply consistent naming standards to migrated assets
6. Request metric extraction for performance-critical queries

Key Platform Differences

Understanding the fundamental differences between Splunk and Dynatrace is critical for a successful migration.

Data Model

Aspect	Splunk	Dynatrace
Storage	Indexes	Buckets (Grail)
Query Language	SPL (Search Processing Language)	DQL (Dynatrace Query Language)
Data Types	Events (logs)	Logs, Metrics, Traces, Events, Entities
Schema	Schema-on-read	Schema-on-read with semantic model

Aspect	Splunk	Dynatrace
Retention	Index-based	Bucket-based with tiered storage

Alerting Model

Aspect	Splunk	Dynatrace
Execution	Scheduled queries	Continuous monitoring
Frequency	User-defined schedule	Every minute by default (configurable Delay, max 60 min)
Evaluation	Single point-in-time	Sliding window
Threshold	Total over period	Per-minute samples
Intelligence	Rule-based	Dynatrace Intelligence powered

Migration Planning

A successful migration follows this general sequence:

Phase 1: Discovery

1. **Inventory existing assets** - Document all dashboards, alerts, reports, and saved searches
2. **Identify data sources** - Map log sources to hosts, clusters, or namespaces
3. **Prioritize by criticality** - Focus on production monitoring first

Phase 2: Data Validation

1. **Verify log ingestion** - Confirm logs are flowing into Dynatrace
2. **Check field mappings** - Ensure required fields are available
3. **Configure ingest rules** - Add any missing log sources

Phase 3: Query Translation

1. **Convert SPL to DQL** - Translate query syntax

2. **Validate results** - Compare output with Splunk
3. **Optimize performance** - Apply DQL best practices

Phase 4: Alert Migration

1. **Choose alert type** - Anomaly Detector or Workflow
2. **Translate thresholds** - Apply conversion formula
3. **Configure notifications** - Route problems through problem-triggered workflows

Phase 5: Dashboard Migration

1. **Recreate visualizations** - Build equivalent charts
2. **Apply filters** - Configure time and segment filters
3. **Validate accuracy** - Compare with Splunk dashboards

Historical Splunk Data

Data already stored in Splunk indexes is **not migrated** into Dynatrace log storage. The migration moves configuration — queries, alerts, dashboards — and Dynatrace holds log data only from the point ingest starts. Replaying a bulk export of Splunk history through log ingest does not work either: *"The event is dropped if it contains a timestamp older than the configured log age limit."*

That limit depends on when the environment was created: *"For environments created after Dynatrace version 1.348 was released, the default log age limit is 72 hours."* Older environments default to 24 hours, which Dynatrace Support can raise to 72 hours on request. Either way the window covers days, not the months or years a Splunk index holds. If you replay recent days to close a gap and cannot confirm your environment's limit, plan for 24 hours.

Plan for history instead:

1. **Keep historical data readable outside Dynatrace** - Keep Splunk searchable, or move the data to an archive, for as long as your audit, compliance or investigation requirements say it must be kept.
2. **Size the parallel run to the look-back you need** - In community practice, teams run both platforms until Dynatrace has accumulated the history its dashboards, alert baselines and the data-driven threshold strategy in S2D-04 depend on — verify the window against your own longest look-back before cutover.

The `splunksearchapireceiver` Collector component, which ships in the Dynatrace Bindplane OTel Collector distribution, exports Splunk search results with each event's original `_time` as the record timestamp, so it is bound by the same age limit and cannot backfill history either.

If your archive is text files in Amazon S3, a Preview can query it in place. SaaS 1.348 (staged tenant rollout from 10/12/2026) introduces *Query external cloud data with DQL (Preview)*. Grail reads files you reference through an External Data Store and parses them at read time with DPL, while the files stay in your bucket. Access is by request. The preview covers Amazon S3 only and text formats only (JSON, NDJSON, CSV, XML and plain text), so check your archive's format against that list before counting on it. A second preview phase, with no start date yet, is described as importing archived S3 logs into Grail with their original timestamps preserved. Until either phase is available to you, the plan above stands: keep history readable where it is.

> **Sources:**
>
> - Log Management and Analytics default limits (DT docs)
> - SaaS 1.348 (DT docs) — *"Rollout start on Oct 12, 2026"*; *"You can now read and analyze data stored in the cloud directly with DQL, providing access to historical data and archives in common formats such as JSON, NDJSON, CSV, XML, or plain text."*
> - Preview program (DT docs) — *"The computation runs in Grail while the data stays in your S3 bucket, so archived data stays queryable without a rehydration step."*; *"The preview is limited to Amazon S3 only, with text-based data formats."*; *"Preview phase 2 (start date to be communicated): Import and process archived logs from S3 into Grail, with their original timestamps preserved"*
> - splunksearchapireceiver at v1.2.0 (Dynatrace GitHub). **Code read (10/29/2026):** `model.go` maps the search result's `_time` to `Time`; `receiver.go` parses it and passes it to `log.SetTimestamp`; `metadata.yaml` lists `distributions: [Dynatrace-Bindplane]`.

Series Roadmap

Notebook	Topic	Description
S2D-01	Getting Started	This notebook - overview and planning
S2D-02	Locating Logs	Validating log data availability

Notebook	Topic	Description
S2D-03	SPL to DQL	Query translation fundamentals
S2D-04	Anomaly Detectors	Translating alerts to continuous monitoring
S2D-05	Workflow Alerts	When and how to use workflow-based alerting
S2D-06	ArrayMovingSum	Handling extended timeframes (>1 hour)
S2D-07	Metric Creation	Creating log metrics via OpenPipeline
S2D-08	Dashboard Migration	Converting Splunk dashboards
S2D-09	Naming Standards	Organization and naming conventions

Quick Start: Verify Your First Log Source

Before diving deeper, let's verify that logs are available in Dynatrace. This query shows log sources and their record counts.

```
// View available log sources and counts
fetch logs, from:-1h
| summarize count = count(), by:{log.source}
| sort count desc
| limit 20
```

Check logs by host name to match Splunk source mapping.

```
// View log counts by host
fetch logs, from:-1h
| summarize count = count(), by:{host.name}
| sort count desc
| limit 20
```

For Kubernetes workloads, check by cluster and namespace.

```
// View log counts by Kubernetes attributes
fetch logs, from:-1h
| filter isNotNull(k8s.cluster.name)
| summarize count = count(), by:{k8s.cluster.name, k8s.namespace.name}
| sort count desc
| limit 20
```

Next Steps

Continue to **S2D-02: Locating Logs in Dynatrace** to learn the complete process for validating log data availability before beginning your migration.

References

- Dynatrace DQL Reference
- anomaly-detection (DT docs)
- Dynatrace Workflows
- Logs (DT docs)

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.